Cove - Silverback security review

Prepared by: Josselin Feist (josselin@seceureka.com)

August 4th, 2025

Prepared For: Sunil Srivatsa - Storm Labs (sunil@storm-labs.xyz)

Executive Summary **4**
Overview 4
Security findings **5**
Event injection due to missing contract validation **6**
Summary 6
Detailed description 6
Recommendations 7
All the actors use the same address 8
Summary 8
Detailed description 8
Recommendations 9
Fix Status 10
Race condition risks 11
Summary 11
Detailed description 11
Recommendations 12
Inconsistencies in filtering the latest event 14
Summary 14
Detailed description 14
Recommendations 15
Fix status 15
Lack of reorg handling 16
Summary 16
Detailed description 16
Recommendations 17
Fix status 17
Coingecko API leak 18
Summary 18
Detailed description 18
Recommendations 19
Fix Status 19
Unclear definitions of CoveState variables 20
Summary 20
Detailed description 20
Recommendations 21
Fix Status 21
Deposit and redeem requests are only tracked on coveUSD 22
Summary 22
Detailed description 22
Recommendations 23

The bot executes the initialization logic for the github action bots 24
Summary 24
Detailed description 24
Recommendations 25
Fix Status 25
Deadcode reduces the code clarity 26
Summary 26
Detailed description 26
Recommendations 26
Fix Status 27
Several untracked TODOs 28
Summary 28
Detailed description 28
Recommendations 30
Fix Status 30
Outdated dependencies 31
Summary 31
Recommendations 32
Fix Status 32
Appendix A: Finding 001 test case 33
Appendix B: semgrep rule 35
Appendix C: Silverback race condition example 36
Appendix D: Cove risks assessment 37
General external risks evaluation 37
Category: Chain Information 37
Category: Logging and alert 38
Category: Trading 39
Category: Others 39
Oracles risks evaluation 40
Chainlink Oracle 43
Pyth 45
ERC4626 47
Curve 48
Appendix E: CI recommendations 49
Appendix F: Architecture recommendations 50
State separation 52
Thread lock 52
Reorg tracking 52
Appendix G: Watching assets 54

Executive Summary

Overview

Storm Labs engaged Josselin Feist to perform a two-week engagement (2025-07-21 - 2025-07-28) to review the silverback bot implementation and perform a general risk assessment.

Over the course of the review, we identified 12 issues, the most critical of which is a high-severity event injection vulnerability caused by missing address validation during event retrieval. Our primary focus was on blockchain-specific logic and architectural risks, with an emphasis on long-term robustness and security. Accordingly, the review placed less emphasis on code correctness, focusing instead on structural improvements.

The codebase is still under active development, with several incomplete features and varying levels of code quality. If greater care is placed on design and testing early on, it will significantly improve maintainability and reduce friction in future development.

The risks assessment (Appendix D) identified several components that can be removed and highlighted components that require strong access controls. Furthermore, the analysis provides an initial framework to evaluate price feeds.

We recommend that Storm Labs:

- Address all the issues identified. We recommend this priority order:
    - Fix critical issue: Issue 1
    - Fix the non-critical issues: issues 2, 4, 9
    - Fix issues associated with a general code cleanup: issues 6,9 and 10,
    - Implement the CI recommendations in Appendix E and Appendix B
    - Fix issues through the code refactoring proposed in Appendix F: issues 3, 5, 7
    - Fix additional non-critical issue: issue 8
    - Prepare the code for future feature cycle: Issue 11, 12 and Appendix G
- Review and maintain the risks assessment provided in Appendix D
- Introduce more frequent feature freeze periods to prioritize code cleanup and increase test coverage.

On September 20, 2025, we reviewed the fixes provided by Storm Labs and updated the report accordingly.

Security findings

ID	Title	Severity	Likelihood	Fix Status
001	Event injection due to missing contract validation	*High*	*High*	*Fixed*
002	All the actors use the same address	*Informational*	*High*	*Acknowledged*
003	Race condition risks	*Low*	*Medium*	*Fixed - review limited*
004	Inconsistencies in filtering the latest event	*Low*	*Low*	*Fixed - review limited*
005	Lack of reorg handling	*Medium*	*Low*	*Fixed - review limited*
006	Coingecko API leak	*Informational*	*Low*	*Fixed*
007	Unclear definitions of CoveState variables	*Informational*	*High*	*Acknowledged*
008	Deposit and redeem requests are only tracked on coveUSD	*Informational*	*High*	*Acknowledged*
009	The bot executes the initialization logic for the github action bots	*Informational*	*High*	*Fixed*
010	Deadcode reduces the code clarity	*Informational*	*High*	*Fixed*
011	Several untracked TODOs	*Informational*	*High*	*Partially Fixed*
012	Outdated dependencies	*Informational*	*High*	*Fixed*

“Fixed - review limited” indicates fixes that led to significant changes that were not covered by this review

Event injection due to missing contract validation			
Severity: High	*Likelihood: High*	*Fix status: Fixed*	*001*

Summary

An attacker can exploit the `LogFilter.from_event`'s lack of contract address validation to force the bot to use arbitrary events.

Detailed description

For several operations, the bot requires to read events emitted using `LogFilter.from_event`:

```
# Create log filter for OrderCreated events
log_filter: LogFilter = LogFilter.from_event(
    event=bm.OrderCreated,
    start_block=start_block,
    stop_block=end_block,
)
```

Figure 1: cove-silverback/cove_submit_cowswap_orders.py#L42-L46

`from_event` has an optional parameter `addresses`, which is used to filter out the events coming only from these addresses:

```
@classmethod
def from_event(
    cls,
    event: Union[EventABI, "ContractEvent"],
    search_topics: Optional[dict[str, Any]] = None,
    addresses: Optional[list[AddressType]] = None,
    start_block=None,
    stop_block=None,
):
```

Figure 2: ApeWorX/ape/ape/types/events.py#L67C7-L67C14

If no address is provided, any event that matches the event's ABI will be returned.

For most of the calls to `from_event`, the parameter is not provided. As a result, an attacker can generate an event that follows the same ABI but with arbitrary values, causing the bot to perform arbitrary actions.

This impacts the following actions:

- Token swap proposal via `propose_token_swap_cli` and `get_last_tokenswap_proposed_event`
- Token swap execution via `execute_token_swap_cli`
- AddRegistry processing via `list_registry_events`
- Rebalance processing via `get_last_proposed_rebalance` and `get_last_rebalance_proposed_event`
- External trades processing via `get_last_executed_external_trades`

Appendix A contains a minimal test case showing how to generate a malicious `RebalanceProposed` event.

Recommendations

Short term, use the `addresses` parameter when calling `LogFilter.from_event` to ensure the events are coming from the right contracts.

Long term, add the semgrep rule provided in Appendix B to the CI.

Fix status

Fixed with PR31. The semgrep rule was also added in PR353.

All the actors use the same address			
Severity: Informational	*Likelihood: High*	*Fix status: Acknowledged*	*002*

Summary

The bot uses the same accounts for the rebalancer and swap proposers and the swap executor. This diverges from the roles separation of the smart contracts, which defines different privileges for the roles.

Detailed description

The Cove smart contracts define fine-grained access controls, with different actors:

```
   /// @notice Rebalance Proposer role. Rebalance proposers can propose
a new rebalance.
   bytes32 private constant _REBALANCE_PROPOSER_ROLE =
keccak256("REBALANCE_PROPOSER_ROLE");
   /// @notice TokenSwap Proposer role. Token swap proposers can propose
a new token swap.
   bytes32 private constant _TOKENSWAP_PROPOSER_ROLE =
keccak256("TOKENSWAP_PROPOSER_ROLE");
   /// @notice TokenSwap Executor role. Token swap executors can execute
a token swap.
   bytes32 private constant _TOKENSWAP_EXECUTOR_ROLE =
keccak256("TOKENSWAP_EXECUTOR_ROLE");
```

Figure 1: src/BasketManager.sol#L35-L40

However, the Silverback bot uses the same account for all actors, where

- `get_cove_bot_sender`,
- `get_cove_rebalance_proposer`,
- `get_cove_token_swap_proposer` and `get_cove_token_swap_executor`

Return the same address (returned by `get_cove_bot_sender`):

```
def get_cove_rebalance_proposer():
   """
   Returns the Cove Bot rebalance proposer account based on the
environment.
   """
   if APP_ENV in (PRODUCTION, STAGING):
```

```
        return get_cove_bot_sender()
    elif APP_ENV == DEVELOPMENT:
        return prank_cove_ops_multisig()
    else:
        raise ValueError("Invalid environment")


def get_cove_token_swap_proposer():
    """
    Returns the token swap proposer account based on the environment.
    """
    if APP_ENV in (PRODUCTION, STAGING):
        return get_cove_bot_sender()
    elif APP_ENV == DEVELOPMENT:
        return prank_cove_ops_multisig()
    else:
        raise ValueError("Invalid environment")


def get_cove_token_swap_executor():
    """
    Returns the Cove Token Swap executor account based on the
environment.
    """
    if APP_ENV in (PRODUCTION, STAGING):
        return get_cove_bot_sender()
    elif APP_ENV == DEVELOPMENT:
        return prank_cove_ops_multisig()
    else:
        raise ValueError("Invalid environment")
```

Figure 2: utils/accounts/account_utils.py#L80-L113

As a result, the account separation created by the contracts is not followed in the current deployment schema.

Recommendations

Short term, create separate accounts to match the contracts setup.

Long term, add tests focussing on the contracts caller access controls.

Fix Status

The issue is tracked on github through PR341 and PR349

Race condition risks			
Severity: Low	*Likelihood: Medium*	*Fix status: review limited*	*003*

Summary

The bot has a fragile architecture against race conditions, with multiple factors increasing the likelihood of their occurrence.

Detailed description

Ape's silverback uses TaskIQ to manage an async task queue. This architecture means that multiple tasks might be executed in parallel. The bot's memory might therefore be accessed by multiple tasks at the same time.

The documentation of the bot's state mentions:

> *You can use `bot.state` to store any python variable type, however note that the item is not networked nor threadsafe so it is not recommended to have multiple tasks write to the same value in state at the same time.*

Figure 1: ApeWorX/silverback//docs/userguides/development.md#bot-state

Cove's bot uses one `CoveState` object which is shared across all the tasks:

```
@bot.on_startup()
@handle_sev_errors(bot)
def bot_startup(startup_state: StateSnapshot):
    ############################
    # Cove basket setup
    ############################
    bot.state.cove = CoveState()
```

Figure 2: ApeWorX/ape/ape/types/events.py#L67C7-L67C14

For example, `cove.sender` might be written when a rebalancer operation occurs and when the target's weights are updated:

```
_bot.state.cove.sender = _bot.state.cove.rebalance_proposer
```

Figure 3: scripts/cove_01_propose_rebalance.py#L144

```
_bot.state.cove.sender = get_cove_bot_sender()
[...]
                tx = strategy.setTargetWeights(
                    update["bit_flag"],
                    update["proposed_weights"],
                    sender=_bot.state.cove.sender,
                )
```

Figure 4: scripts/cove_05_set_target_weights.py#L57-L334

If both tasks are executed at the same time, the following order of events might occur:

1. `Write get_cove_bot_sender() (target's weight operation)`
2. `Write _bot.state.cove.rebalance_proposer (rebalancer operation)`
3. `Read (target's weight operation)`

Thereby, when the read happens, the target's weight operation will use the value written by the rebalancer operation *(note due to issue 002, the same value would be returned in this case)*.

There are multiple factors that can lead tasks from being executed concurrently:

- Some operations might be delayed (e.g. due to `time.sleep` in `send_private_transaction`)
- The decorators `@bot.on_` and `@bot.cron` might be triggered at the same time
- Multiple events within the same block will trigger `@bot.on_` multiple times

Appendix C contains a minimal test case showing how two `@bot.cron` tasks can be perturbed by using the same state.

Note: if silverback is launched with worker (`silverback worker`) instead of standard run (`silverback run`), the bot will be multi-processed, which would impact the design of the bot.

As a result of this review, Improve documentation around race condition was opened on ape's github.

Recommendations

Short term, use `asyncio.Lock` to implement a mutex between the different operations.

Long term, consider creating different `CoveState` objects per action type, to further isolate the underlying data.

Fix status

A reentrant lock was implemented through PR317 and PR323. Dedicated type were also added in PR325. The full extent of the code refactored was not reviewed.

Inconsistencies in filtering the latest event			
Severity: Low	*Likelihood: Low*	*Fix Status: review limited*	*004*

Summary

`Chain.provider.get_contract_logs` is not used consistently across the codebase to ensure that the calling function returns the latest matching event.

Detailed description

`Get_last_rebalance_proposed_event` is meant to return the most recent `RebalanceProposed` event:

```
def get_last_rebalance_proposed_event(
    bm: Contract,
) -> ContractLog | None:
    """Get the most recent RebalanceProposed event based on the current
rebalance
    status timestamp.

    Args:
        bm: The BasketManager contract instance

    Returns:
        Optional[ContractLog]: The most recent RebalanceProposed event,
or None
        if no event found
    """
    rebalance_status = bm.rebalanceStatus()
    proposal_block =
get_block_number_from_timestamp(rebalance_status.proposalTimestamp)

    log_filter = LogFilter.from_event(
        event=bm.RebalanceProposed,
        start_block=proposal_block,
        stop_block=proposal_block + 1,
    )
```

```
    logs = list(chain.provider.get_contract_logs(log_filter))
    return logs[0] if len(logs) > 0 else None
```

Figure 1: utils/contracts/basket_manager.py#L992-L1012

However the first element of the logs list (`logs[0]`) is returned, which might not be the most recent event.

Across the codebase, three patterns are used to select the most recent event:

- `Get_last_rebalance_proposed_event`
  - Returns the first event (`logs[0]`)
- `Get_last_proposed_rebalance, get_last_executed_external_trades and get_last_tokenswap_proposed_event`
  - Return the latest event on the list (`logs[-1]`)
- `Propose_token_swap_cli` and `execute_token_swap_cli`
  - Order the events by block number, and return the latest one

As a result, if multiple events match (e.g. due to a bug, or a re-org) the wrong event might be selected.

Recommendations

Short term, order the log by combining the block number, transaction number and index number. Consider also checking if the removed flag is set, and discard removed logs *(note: this is not easily done within ape at the moment and would require the framework to track the removed attribute)*.

Long term, create coding guidelines for adding new tasks/actions to the bot. The coding guidelines should include best practices and code snipped/internal functions that can be reused.

Fix status

Fixed in PR315, the client provided this detail

> *Created unified event handling utilities in `utils/contracts/event_utils.py` with `get_latest_event()` and `get_events_in_range()` functions. Events are now sorted deterministically by block number, transaction index, and log index to handle multiple events in the same block. Added filtering for removed logs during reorgs (`removed=False`) and proper handling for missing transaction indices with warning logs. All event queries now use consistent ordering logic preventing stale event processing.*

The full extent of the code changes were not reviewed.

Lack of reorg handling			
Severity: Medium	*Likelihood: Low*	*Fix status: review limited*	*005*

Summary

The bot currently works from the latest blocks, which might not be included in the finalized state of the chain.

Detailed description

Ethereum's blocks finality can be either

- Earliest
- Pending
- Latest
- Safe
- Finalized

The cove's bot uses "latest" blocks when querying the node provider. For example, in `check_oracle_prices:`

```
block = networks.provider.get_block("latest")
```

Figure 1: bots/cove_bot.py#L385

This also happens in:

- `check_rebalance_status`
- `check_price_per_share_increase`
- `propose_rebalance`

The same behavior is also present for actions performed through the `on_` decorator, as ape relies on `eth_subscribe`, which is not an standardized API (execution-apis/496). Most providers will re-emit blocks in case of reorg:

> In case of a chain reorganization the subscription will emit the last header in the new chain.
>
> Therefore the subscription can emit multiple headers on the same height.

Figure 2: https://geth.ethereum.org/docs/interacting-with-geth/rpc/pubsub

The bot has no logic to handle a re-org. As a result, a re-org might lead to

- Reusing the same event multiple times (e.g. a pending deposit might appear twice in the log),
- Performing actions based on a state that will not be included in the finalized state (e.g.: triggering a rebalancer on a basket that does not need rebalancing)
- Updating `CoveState` incorrectly (e.g. by setting `last_weight_update_check_time`)

As a result of this review, Improve documentation around re-org risks, and Add support for the removed attribute in contractLog were opened on ape's github.

Recommendations

All the calls to `get_block` can use "finalized" to ensure the block finality, but correctly handling re-org through ape's `on_` decorator would require significant changes, as it relies on `eth_subscribe`, which does not offer a standardized way to only subscribe to finalized blocks.

Instead of mitigating re-org, another approach is to detect them and require manual intervention when they impact the system. Consider implementing the architectural recommendation in Appendix F to ease the detection of re-org.

Given the current state of the codebase, we recommend favoring the detection of re-org, and to evaluate a more robust solution once the bot's codebase has been stabilized.

Fix status

The code was refactored to detect and handle reorg in PR319. The full extent of the code refactored was not reviewed.

Coingecko API leak			
Severity: Informational	*Likelihood: Low*	*Fix Status: Fixed*	*006*

Summary

A logging message leaks the coingecko's API key.

Detailed description

The coingecko api is called through a GET request. The url contains the API key (`request_url`):

```
   request_url =
f"{COINGECKO_API_URL}?x_cg_demo_api_key={api_key}&vs_currencies=usd&ids=
{ids}"

   logger.info(
       "Querying CoinGecko API for token prices",
       context={
           "num_tokens": len(coingecko_ids),
       },
   )

   response = retry_get(request_url)
```

Figure 1: utils/network/requests_utils.py#L197-L206

All the GET requests url are logged:

```
def retry_get(url: str, raise_for_status: bool = True, **kwargs) ->
requests.models.Response:
   """
   Retry a GET request up to 5 times with exponential backoff.
   """
   # Remove raise_for_status from kwargs if it was passed in
   kwargs.pop("raise_for_status", None)

   try:
```

```
        logger.debug(f"Making GET request to {url}", context={"params":
kwargs.get("params")})
```

Figure 2: utils/network/requests_utils.py#L28-L36

As a result, the log will contain the API key.

This issue was classified as informational given that the relevant function is never used (see issue 9)

Recommendations

Short term, remove `query_coingecko_prices` and `COINGECKO_API_KEY`

Long term, update `retry_get` to allow url parameters for not being logged.

Fix Status

Fixed with PR316

Unclear definitions of CoveState variables			
Severity: Informational	*Likelihood: High*	*Fix Status: Acknowledged*	*007*

Summary

The state of the Cove bot is mangled between a `CoveState` objects and `SimpleNamespace` objects, which makes the review and the maintenance of the cove variables error prone

Detailed description

The silverback bot is used in two context:

- As an ongoing bot (defined in bots/cove_bot.py)
- As a one-time execution bot, triggered by a github actions (defined in bots/cove_*.py)

When `cove_bot` is executed, the bot's state is saved in a `CoveState` object:

```
bot.state.cove = CoveState()
```

Figure 1: bots/cove_bot.py#L150

When the bot is executed through a github actions, bot.state is a `SimpleNamespace:`

```
bot.state = SimpleNamespace()
bot.state.boosties = SimpleNamespace()
bot.state.boosties.pauser = get_boosties_bot_sender()
```

Figure 2: scripts/cove_01_propose_rebalance.py#L25-L27

However both setup use the same function calls. Some of the variables used are not defined in the CoveState object, for example:

```
       _bot.state.cove.rebalance_status =
RebalanceStatus.REBALANCE_PROPOSED
       _bot.state.cove.baskets_to_rebalance =
rebalance_proposed_event["baskets"]
       _bot.state.cove.target_weights =
rebalance_proposed_event["proposedTargetWeights"]
       _bot.state.cove.basket_assets =
rebalance_proposed_event["basketAssets"]
```

Figure 3: scripts/cove_01_propose_rebalance.py#L235C60-L238

While python allows for dynamic attribute assigning, which allows `CoveState` to take the same attribute as the `SimpleNamespace`, doing so makes the code more difficult to review and maintain:

- `CoveState` might have additional logic not present in `SimpleNamespace`
- The overall variables usage is difficult to track
- The python typing hint will not work properly

Recommendations

Review and consider implementing the architectural recommendations provided in Appendix F.

Fix Status

The issue was acknowledged in 342.

Deposit and redeem requests are only tracked on coveUSD

Severity: Informational	*Likelihood: High*	*Fix Status: Acknowledged*	*008*

Summary

Currently the bot only tracks deposit and redeem requests made on the coveUSD's basket token, and does not record the requests made to the other basket tokens.

Detailed description

The functions handling the deposit and redeem requests use the `bot.on_()` decorators:

```
@bot.on_(coveUSD.DepositRequest)
@handle_sev_errors(bot)
def handle_deposit_request(log: ContractLog):
    """
    Handle DepositRequest event from CoveUSD.
    """
    record_deposit_request(bot, coveUSD.address, log)


@bot.on_(coveUSD.RedeemRequest)
@handle_sev_errors(bot)
def handle_redeem_request(log: ContractLog):
    """
    Handle RedeemRequest event from CoveUSD.
    """
    record_redeem_request(bot, coveUSD.address, log)
```

Figure 1: bots/cove_bot.py#L976-L991

However this is only set for the coveUSD basket token:

```
coveUSD = Contract(COVE_USD, abi="abis/BasketToken.abi.json")
```

Figure 2: bots/cove_bot.py#L70C1-L70C62

Therefore all the other basket tokens are currently not tracked. And if the bot is deployed with another basket manager, no basket token will be tracked.

Recommendations

Short term, track all the basket tokens for the basket manager.

Long term, increase the test coverage to include multiple basket tokens.

Fix Status
Acknowledged in 343.

The bot executes the initialization logic for the github action bots

Severity: Informational	*Likelihood: High*	*Fix Status: Fixed*	*009*

Summary

Due to the overall use of module level variables and execution, the bot initialization executes the code meant for the github action scripts. While none of the executions are currently impacting the bot, further development might introduce issues.

Detailed description

The silverback bot is used in two context:

- As an ongoing bot (defined in bots/cove_bot.py)
- As a one-time execution bot, triggered by a github actions (defined in bots/cove_*.py)

Some of the ongoing bot functions are defined in the one-time execution bot module;:

```
from scripts.check_bot_balance import check_bot_balance
from scripts.cove_01_propose_rebalance import propose_rebalance
from scripts.cove_02_propose_token_swap import
propose_token_swap_from_event
from scripts.cove_04_complete_rebalance import complete_rebalance
from scripts.cove_05_set_target_weights import set_target_weights
from scripts.cove_06_operator_deposit_redeem import
sweep_pending_requests
```

Figure 1: bots/cove_bot.py#L10-L15

All the scripts module have module-level assignment and function calls, for example:

```
# Bot state setup
bot = SimpleNamespace()
bot.state = SimpleNamespace()
bot.state.cove = SimpleNamespace()
init_script_logging(bot.state, BotType.COVE, "propose_rebalance")
```

Figure 2: scripts/cove_01_propose_rebalance.py#L28

This logic will be executed from the functions are imported:

> ## 6.1. More on Modules
>
> A module can contain executable statements as well as function definitions. These statements are intended to initialize the module. They are executed only the *first* time the module name is encountered in an import statement. [1] (They are also run if the file is executed as a script.)

Figure 3: https://docs.python.org/3/tutorial/modules.html#more-on-modules

The `bot` variable is confined to the module's scope, and `init_script_logging` will not directly impact the ongoing bot actions. However this pattern is likely to cause issues as the bot evolves and new module level actions are added to the modules.

Recommendations

Short term, move all the functions from the scripts modules to separate modules, without module-level assignment or function calls.

Long term, minimize module-level assignments and function calls by wrapping the logic into internal functions.

Fix Status

Fixed in PR320.

Deadcode reduces the code clarity			
Severity: Informational	*Likelihood: High*	*Fix Status: Fixed*	*010*

Summary

The codebase contains several functions that are never called, increasing the code's complexity and maintenance cost.

Detailed description

The following functions are never called:

- `run_every_n_blocks`
- `get_target_weights_for_bitflag`
- `get_last_updated_for_bitflag`
- `supports_bitflag`
- `run_every_n_blocks`
- `query_coingecko_prices`
- `get_all_onchain_prices`
- `initialize_bot_state`
- `generate_error_selector`

The following classes are never used

- `GasPriceTooHighError`
- `SwapAdapterType`

The following variables are never used:

- `keys.ENVIRONMENT`
- `threshold.SCRIPTS_CONFIG`

Dead code increases the code review effort and makes code maintenance harder. Moreover, given that python is a non-compiled code, it is likely that the unused and untested code does not stay in synchronization with the rest of the codebase as it evolves.

Recommendations

Short term, remove all the functions listed above

Long term,

- Increase the test coverage and ensure that every function is tested
- Run vulture periodically on the codebase (`vulture . --exclude test_*py`)

Vulture might trigger several false alarms, in particular due to the use of the bot's decorator, and it is better utilized as a periodic tool rather than a CI tool.

Fix Status

Fixed with PR314 and PR309 (Boosties removal).

Several untracked TODOs			
Severity: Informational	*Likelihood: High*	*Fix Status: Partially fixed*	*011*

Summary

The codebase contains several TODOs, and it is unclear which ones are going to be addressed.

Detailed description

The code contains the following TODOs left:



Figure 1: Existing TODO

Moreover, some features are not implemented, and are also not marked as TODO, such as:

```
   # Initialize MasterRegistry contract
   # Note: You'll need to replace this with the actual MasterRegistry
address
   master_registry =
Contract("0x0000000000000000000000000000000000000000",
abi="abis/MasterRegistry.abi.json")
```

Figure 2: scripts/cove_list_master_registry_events.py#L82-L84

These features are not tracked in the issues tracker, making their planning more difficult.

Recommendations

Short term, address all the TODO left, or link them to github issues.

Long term, periodically perform features freeze and use the period to identify the missing features and prioritize their implementation.

Fix Status

Partially fixed. Several TODOs were cleaned up in PR337, however several TODOs still appear to be present in the codebase, and aren't tracked in the issue tracker (e.g. in `weight_strategy.py`)

Outdated dependencies

Severity: Informational	*Likelihood: High*	*Fix Status: Fixed*	*012*

Summary

Several dependencies are out of date:

Package	Current	Latest
aiohttp	3.12.14	3.12.15
asttokens	2.4.1	3.0.0
base58	1.0.3	2.1.1
bitarray	3.5.2	3.6.0
click	8.1.8	8.2.1
cyclonedx-python-lib	9.1.0	11.0.0
dataclassy	0.11.1	1.0.1
eth-keyfile	0.8.1	0.9.1
ipython	8.37.0	9.4.0
izulu	0.50.0	0.55.0
numpy	1.26.4	2.3.2
opentelemetry-api	1.35.0	1.36.0
opentelemetry-sdk	1.35.0	1.36.0
packaging	23.2	25.0
pulp	3.2.1	3.2.2
py-geth	5.6.0	6.1.0
py-multihash	0.2.3	2.0.1
pydantic-core	2.33.2	2.37.2

regex	2024.11.6	2025.7.31
rich	13.9.4	14.1.0
sqlalchemy	2.0.41	2.0.42
termcolor	2.3.0	3.1.0
watchdog	3.0.0	6.0.0

Recommendations

Short term, consider updating the dependencies. Make sure to update the uv.lock alongside to pin the versions.

Long term, integrate pip-audit and dependabot to the CI.

Fix Status

The dependencies were updated in PR305 (and PR311 for pagerduty). Note: we did not review the full extent of the dependency updates.

Appendix A: Finding 001 test case

The following contains a minimal test case to demonstrate finding 001:

- The bot in Figure 1 simply listens to the event `RebalanceProposed`
- The malicious contract in Figure 2 emits an event that match `RebalanceProposed`'s abi

You can run the bot, and deploy the contract on an Anvil's based chain:

```
$ anvil
$ forge create src/Fake.sol:Fake  --private-key <key generated by anvil>
```

And after one minute, the fake `RebalanceProposed` event will be printed by the bot.

```python
from silverback import SilverbackBot
from ape import Contract, chain
from ape.types.events import LogFilter

bot = SilverbackBot()

BASKET_MANAGER = "0x716c39658Ba56Ce34bdeCDC1426e8768E61912f8"

@bot.cron("* * * * *")
def test(timestamp):
   bm = Contract(BASKET_MANAGER, abi="abis/BasketManager.abi.json")
   current_block = chain.blocks.height

   # Create log filter for RebalanceProposed events
   log_filter = LogFilter.from_event(
       event=bm.RebalanceProposed,
       start_block=0,
       stop_block=current_block,
   )

   # Get logs from provider and convert to list
   logs = list(chain.provider.get_contract_logs(log_filter))

   print(logs)
```

Figure 1: simplified bot

```
contract Fake{


   event RebalanceProposed(
       uint40 indexed epoch,
       address[] baskets,
       uint64[][] proposedTargetWeights,
       address[][] basketAssets,
       bytes32 basketHash
   );

   constructor() public{
       address[] memory baskets;
       uint64[][] memory proposedTargetWeights;
       address[][] memory basketAssets;
       bytes32 basketHash;
       emit RebalanceProposed(0x41, baskets, proposedTargetWeights,
basketAssets, basketHash);
   }

}
```

Figure 2: Fake contract

Appendix B: semgrep rule

The following semgrep rule can be used to detect issue 001. To run it, save the rule in a yaml file and run

- *semgrep --config <file>.yaml*

```
rules:
 - id: logfilter-from-event-missing-addresses
   patterns:
     - pattern: |
         LogFilter.from_event(
           ...
        )
     - pattern-not: |
         LogFilter.from_event(
           ...,
           addresses=...,
           ...
        )
     - pattern-not: |
         LogFilter.from_event(addresses=..., ...)
   message: "LogFilter.from_event called without an 'addresses'
parameter."
   severity: WARNING
   languages: [python]
```

Figure 1: semgrep rule

Appendix C: Silverback race condition example

The following bot shows how two tasks might corrupt their underlying data (see issue 3):

- `Bot.cron` creates a task every minute
- The task picks a random number, and waits 2 minutes

Given that the tasks are executed in parallel, the second time the random number is printed, it will have been modified by another task

```
import time
import random

from silverback import SilverbackBot

bot = SilverbackBot()


@bot.cron("* * * * *")
def handle_new_block(block):

    random_number = random.randint(0, 10**10)
    print(f"Start the wait: random: {random_number}")
    bot.state.random_number = random_number
    time.sleep(120)
    print(f"Wait ended. Original number: {random_number}, current number
{bot.state.random_number}")
```

Figure 1: Bot example

For example:

- Task 1: `Start the wait: random: 10`
- Task 2: `Start the wait: random: 20`
- Task 1: `Wait ended. Original number 10, current number 20`

Appendix D: Cove risks assessment

General external risks evaluation

The following highlights external components, associated risks, and potential improvements. The goal of this section is to help the Storm Labs team identify latent risks.

Overall we recommend to

- **Reduce the chain information providers**, as many are not needed for the bot
- **Review periodically the access controls on the logging services and github actions**, as they give access to sensitive information and actions

The analyses does not include general resilience related risks (e.g. outage), API key leakage, and assume the service providers do not act maliciously.

Category: Chain Information

Name	**Role in System**	**Fallback**	**Security risks**	**API Key**	**Priority**
Infura	Needed for ape-infura	Unused		WEB3_INFURA_API_KEY	Low
Etherscan	Needed for ape-etherscan	Unused		ETHERSCAN_API_KEY	Low
Gauntlet Pivot API	Target weight updates	Mock data (mock_gauntlet_response) Grafana logging	Incorrect weights	GAUNTLET_PIVOT_API_KEY	Medium
CoinGecko	Price oracle	Unused		COINGECKO_API	Low

				_KEY	
Quicknode	Silverback network bot provider Sending private transactions			QUICKNODE_SUBDOMAIN QUICKNODE_AUTH_TOKEN	Low
Alchemy	Silverback network bot provider Sending private transactions	Unused			Low

We recommend to

- Remove the infura, etherscan and coinkego, Alchemy API key, related code usage and dependencies
    - Key: WEB3_INFURA_API_KEY, ETHERSCAN_API_KEY, COINGECKO_API_KEY
    - Function: query_coingecko_prices, and Alchemy's part in `send_private_transaction`
    - Dependencies: ape-infura and ape-etherscan
- Consider logging Gauntlet Pivot response to discord
- Consider removing the mock data for gauntlet's fallback

Category: Logging and alert

Name	**Role in System**	**Fallback**	**Security risks**	**API Key**	**Priority**
Discord bot	Public logging	Grafana logging	Missed log Leak of sensitive information Malicious url/phishing	DISCORD_BOT_TOKEN	Medium
Grafana	Logging		Leak of sensitive information	GRAFANA_RW_API_KEY	Medium
PagerDuty	Incident notification and motivation	Grafana logging	Incident not notified	PAGERDUTY_EVENTS_ROUTING_KEY	**High**

We recommend to

- Periodically review the access controls for Grafana and review the log to identify potential leak of sensitive information
    - In particular the systematic usage of `str(e)` on arbitrary exception handling might lead to leak sensitive information to grafana
- Migrate to the latest version of PagerDuty (see pdpyras migration guide)
- Perform periodic wargame to stress test the logging and incident response plan (see Seal Wargame)

Category: Trading

Name	Role in System	Fallback	Security risks	API Key	Priority
CowSwap	Swap		Malicious or buggy swap solver		**High**
Flashbot	Sending private transactions		MEV / bid collusion		**High**

We recommend to

- Evaluate alternative swappers and solver solutions to increase the swapping resilience

Category: Others

Name	Role in System	Fallback	Security risks	API Key	Priority
Github action	Trigger bot's actions		Trigger malicious operations		**High**
Amazon services	KMS, Secret Manager, SSM, S3, CloudWatch, CodeDeploy		Leak of sensitive information/wallet	USER_HOME/.aws/credentials	**High**

We recommend to

- Periodically review the github access to ensure only authorized users can trigger the bot's actions
- Perform a dedicated operational security review of the amazon services
- Consider merging SSM with Secret manager to reduce the system's complexity

Oracles risks evaluation

Cove relies on multiple Oracles types, each one with benefits and limitations. The oracles can be chained together with `CrossAdapter`.To increase the confidence, Cove uses an `AnchoredOracle` adapter, which uses a primary oracle and a secondary oracle, the secondary providing a min/max bound for the primary Oracle.

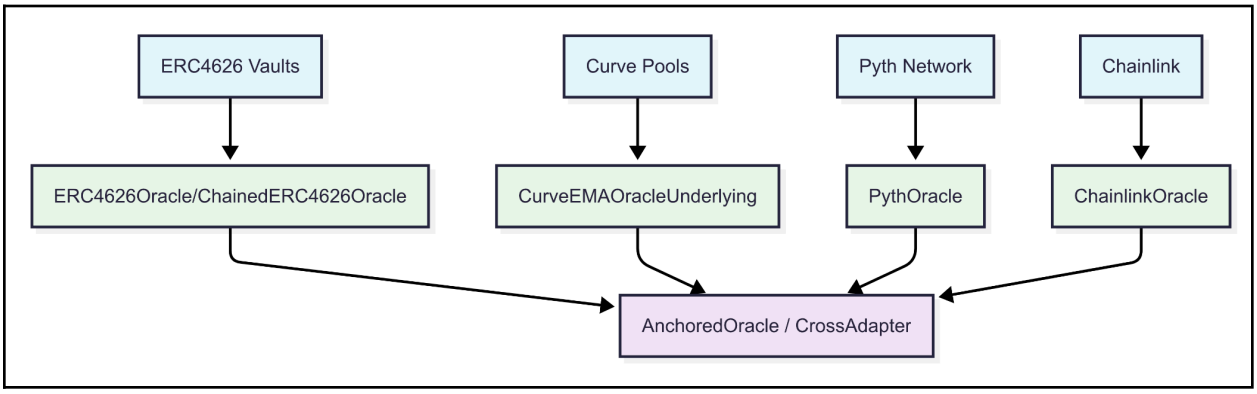


Figure 1: Cove's oracles integration

On one hand, the variety of Oracle types and the usage of the `AnchoredOracle` increase the resilience of the system, but on the other hand, it increases the attack surface and the underlying complexity.

For example, Figure 2 shows the Oracle dependencies of the setup in the deployment script (Deployments_Production.s.sol)

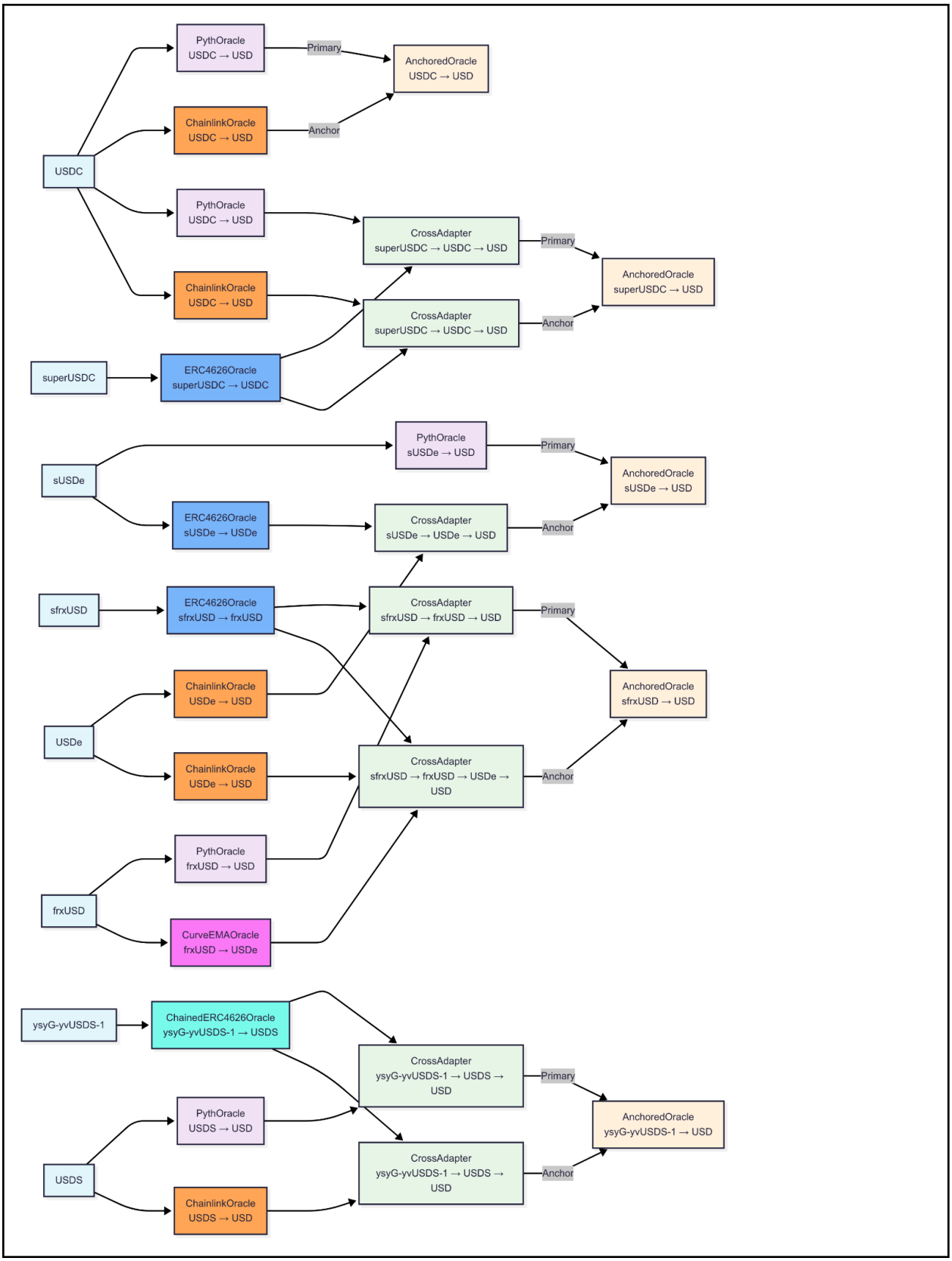


Figure 2: Oracle dependencies in [Deployments_Production.s.sol]

The `AnchoredOracle` can be group in four categories (more details are provided in the following sections)

Paths	**Primary risks**	**Anchor risks**
USDC -> USD	Robust	Robust
sUSDe -> USD	Robust	ERC4626 risks
superUSDC -> USD sfrxUSD -> USD	ERC4626 risks	ERC4626 risks *CurveEMA risks (*)*
ysyG-vUSDS-1	Chained ERC4626 risks	Chained ERC4626 risks

(*) The Storm labs team mentioned that CurveEMA might be replaced by the recent equivalent chainlink price feed

We recommend to:

- **Review and maintain the list of Oracle and their associated risk in a centralized location**
- **Create guidelines on how to use `AnchoredOracle` and `CrossAdapter` based on the underlying risk evaluation of each Oracle.**
- **If deployed in L2, consider also factoring the sequencer uptime** (see Chainlink's recommendations)

The following sections contain detailed analysis of each oracle type. To avoid duplication, these sections do not include smart contracts bugs or general contract integration risks, which are assumed to be handled by the `EulerRouter` (e.g. Chainlink's stateless check).

Chainlink Oracle

While chainlink price feeds are generally considered trusteed Oracles, not all the price feeds have the same quality.

Chainlink provides a six-tier evaluation risk per feed that evaluates uncertainty or volatility of a given feed. In addition, each feed has a different number of minimum Oracle requirements.

The following table is built from the price feed present in the test folder of Cove:

Feed	Used in Prod	Address	Market Risk	Minimum/Max Oracle
usdc-usd	x	0x8fFfFfd4AfB6115b954Bd326cbe7B4BA576818f6	Low Market Risk	11/16
usde-usd	x	0xa569d910839Ae8865Da8F8e70FfFb0cBA869F961	Low Market Risk	11/16
usds-usd	x	0xfF30586cD0F29eD462364C7e81375FC0C71219b1	Medium Market Risk	11/16
dai-usd		0xAed0c38402a5d19df6E4c03F4E2DceD6e29c1ee9	Low Market Risk	13/19
frax-usd		0xB9E1E3A9feFf48998E45Fa90847ed4D467E8BcfD	Medium Market Risk	13/19
eth-usd		0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419	Low Market Risk	21/31
susde-usd		0xFF3BC18cCBd5999CE63E788A1c250a88626aD099	Medium Market Risk	11/16
weeth-eth		0x5c9C449BbC9a6075A2c061dF312a35fd1E05fF22	Medium Market Risk	11/16
ezeth-eth		0x636A000262F6aA9e1F094ABF0aD8f645C44f641C	High Market Risk	11/16

reth-eth		`0x536218f9E9Eb48863970252233c8F271f554C2d0`	High Market Risk	11/16
wbtc-btc		`0xfdFD9C85aD200c506Cf9e21F1FD8dd01932FBB23`	Medium Market Risk	11/16
tbtc-usd		`0x8350b7De6a6a2C1368E7D4Bd968190e13E354297`	Medium Market Risk	11/16
gho-usd		`0x3f12643D3f6f874d39C2a4c9f2Cd6f2DbAC877FC`	Medium Market Risk	11/16
cbbtc-usd		`0x2665701293fCbEB223D11A08D826563EDcCE423A`	Low Market Risk	11/16

Notes:

- No chainlink price feed was set for sdai/usd (Constants.t.sol#L80)
- rsETH/ETH 0x03c68933f7a3F76875C0bc670a58e69294cDFD01 does not appear to be a valid price feed

We recommend to:

- **Not use New Token Feeds, Custom Feeds, or Deprecating feeds** (see selecting-data-feeds#data-feed-categories)
- **Avoid high market risk feed**
- **Avoid chaining medium+ market risks together in a `CrossAdapter`**
- **Avoid Oracle that have less than 11 Oracle minimal requirement**

Pyth

Pyth price feeds come with an inbuilt confidence metric, which is checked on-chain by the contact.

One metric to consider one integrating Pyth price feeds is their number of publishers. The following table is built from the price feed present in the test folder of Cove:

Feed	Used in Prod	Address	# Publishers
USDC/USD	**X**	0xeaa020c61cc479712813461ce153894a96a6c00b21ed0cfc2798d1f9a9e9c94a	32
USDS/USD	**x**	0x77f0971af11cc8bac224917275c1bf55f2319ed5c654a1ca955c82fa2d297ea1	32
frxUSD/USD	**x**	0x7c53208632935ba5122c3cf65a0f4b3e72ba4955b49ad6ba0acf3d9ba405aef3	16
DAI/USD		0xb0948a5e5313200c632b51bb5ca32f6de0d36e9950a942d19751e833f70dabfd	32
SDAI/USD		0x710659c5a68e2416ce4264ca8d50d34acc20041d91289110eea152c52ff3dc39	27
FRAX/USD		0x735f591e4fed988cd38df74d8fcedecf2fe8d9111664e0fd500db9aa78b316b1 (*)	30
USDE/USD		0x6ec879b1e9963de5ee97e9c8710b742d6228252a5e2ca12d4ae81d7fe5ee8c5d	32
ETH/USD		0xff61491a931112ddf1bd8147cd1b641375f79f5825126d665480874634fd0ace	33
SUSDE/USD		0xca3ba9a619a4b3755c10ac7d5e760275aa95e9823d38a84fedd416856cdba37c	29
WEETH/USD		0x9ee4e7c60b940440a261eb54b6d8149c23b580ed7da3139f7f08f4ea29dad395	32
RETH/ETH		0xa0255134973f4fdf2f8f7808354274a3b1ebc6ee438be898d045e8b56ba1fe13	27

wBTC/USD		0xc9d8b075a5c69303365ae23633d4e085199bf5c520a3b90fed1322a0342ffc33	32

Notes:

- FRAX USD was 0xc3d5d8d6d17081b3d0bbca6e2fa3a6704bb9a9561d9f9e1dc52db47629f862ad in the test, but it should be 0x735f591e4fed988cd38df74d8fcedecf2fe8d9111664e0fd500db9aa78b316b1 (Constants.t.sol#L83)
- ezETH/ETH and rsETH/ETH, GHO/USD and cbBTC/USD have these TODOs:

```
// ezETH/ETH
// TODO: pyth price feed not found
```

- The wBTC/USD pair was marked in the wBTC/BTC section:

```
// wBTC/BTC
bytes32 public constant PYTH_WBTC_USD_FEED =
0xc9d8b075a5c69303365ae23633d4e085199bf5c520a3b90fed1322a0342ffc33;
```

We recommend maintaining the list of Oracle and their associated number of publishers, and avoid Oracle with a lower number of publishers.

ERC4626

ERC4626 Oracle are generally more risky, and only meant to be used if no other price feed are available, as stated in the documentation:

```
/// This oracle relies on the convertToAssets/convertToShares functions
of the underlying ERC4626 vault.
/// If the dependent ERC4626 contract does not implement sufficient
protection against donation attacks,
/// sudden price jumps may occur when large amounts of assets are
donated to the vault without a proportional
/// increase in shares. Users should verify the security measures
implemented by the underlying vault.
/// Due to this risk, this oracle should only be used when there is no
direct price feed available for the vault token.
```

Figure 3: src/oracles/ERC4626Oracle.sol#L13-L22

The following ERC4626 Oracle are used:

- ETH_SUPERUSDC (0xF7DE3c70F2db39a188A81052d2f3C8e3e217822a)
  - Asset USDC: (0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48)
- ETH_SUSDE (0x9D39A5DE30e57443BfF2A8307A4256c8797A3497)
  - Asset USDe (0x4c9EDD5852cd905f086C759E8383e09bff1E68B3)
- ETH_SFRXUSD (0xcf62F905562626CfcDD2261162a51fd02Fc9c5b6)
  - Asset frxUSD (0xCAcd6fd266aF91b8AeD52aCCc382b4e165586E29)

And the following ERC4626 Oracles are used and chained:

- ETH_YSYG_YVUSDS_1 (0x81f78DeF7a3a8B0F6aABa69925efC69E70239D95)
  - Asset yG-yvUSDS-1 (0xd57aEa3686d623dA2dCEbc87010a4F2F38Ac7B15)
  - Asset USDS-1 yVault (0x182863131F9a4630fF9E27830d945B1413e347E8)
  - Asset USDS (0xdC035D45d973E3EC169d2276DDab16f1e407384F)

A python price feed path exists from SFRXUSD to USD:

- SFRXUSD/FRXUSD (17 publishers)
- FRXUSD/USD (16 publishers)

We recommend to

- **Use the recent SFRXUSD to USD path in pyth instead of the ERC4626 Oracle**
- **Avoid using ERC4626 oracle when possible**
- **Not using ERC4626 oracle as the primary source for an `AnchoredOracle`**

Curve

The Curve Oracle documentation has the following warning:

```
   /// IMPORTANT: The isBaseUnderlying and isQuoteUnderlying flags must
match the pool's actual asset types.
   /// Curve pools with ERC4626-compatible tokens (like sUSDE/sfrxUSD)
internally handle underlying asset conversions.
   /// The price_oracle() function returns the price in terms of the
underlying assets (e.g., USDE/frxUSD).
   /// This behavior is pool-specific and depends on the pool's
initialization parameters.
   ///
   /// WARNING: Incorrect configuration of isBaseUnderlying and
isQuoteUnderlying will result in
   /// incorrect price calculations. Always verify the pool's asset
types off-chain before deployment.
   ///
   /// WARNING: Some StableSwap-NG pools deployed before Dec-12-2023
have a known oracle vulnerability.
   /// See
(https://docs.curve.fi/stableswap-exchange/stableswap-ng/pools/oracles/#
price-oracles) for more details.
   /// Additionally, verify that the pool has enough liquidity before
deploying this adapter.
```

Only one Curve oracle is used in production for frxUSD/sUSDe (0x3BD1017929b43c1414bE2Aca39892590fBa4d6e2), which follows the recommendations provided in the documentation.

We recommend to:

- **Review each curve pool to ensure they match the Oracle documentation warnings**
- **Avoid curve Oracle for pool with low liquidity**

Note: during the review, the Storm labs team mentioned that a chainlink feed is now available for the pair frsUSD -> USD: https://data.chain.link/feeds/ethereum/mainnet/frxusd-usd, which might allow to remove the Curve Oracle

Appendix E: CI recommendations

The following contains recommendations of linters and static analyzer that Storm labs could consider for its CI:

- **Mypy**. When developing production python code, type annotation should be considered as non-optional.
    - Several types are incorrectly defined in the codebase
- **Pip-audit.** Pip-audit will warn if any vulnerabilities is detected in a dependency
- **Dependabot.** Dependabot will automatically create PRs when a dependency needs to be updated
- **Zizmor**. Zizmor will help catching issues in the Github Actions
    - For example, zizmor will highlight actions that should have a pin to a hash to avoid dependencies issues
- **Semgrep.** Semgrep is a security scanner that will catch the most common vulnerabilities. It can be also customized with new rules (see Appendix B)
    - For example, semgrep will warn about a variable interpolation risk in github action, while the warning is currently not exploitable, it highlights improvement to be made in the action

Appendix F: Architecture recommendations

The current bot's architecture relying on a single state for the tasks (`CoveState`) has several limitations, including:

- No protection against race condition (see issue 3)
- Unclear separation between the variables usage (see issue 7)
- No mechanism to track re-org (see issue 5)
- No mechanism to track different token basket and basket manager

To alleviate that, we propose a redesign of the CoveState object, to split it into multiple classes, where each class is guarded with a mutex/lock.

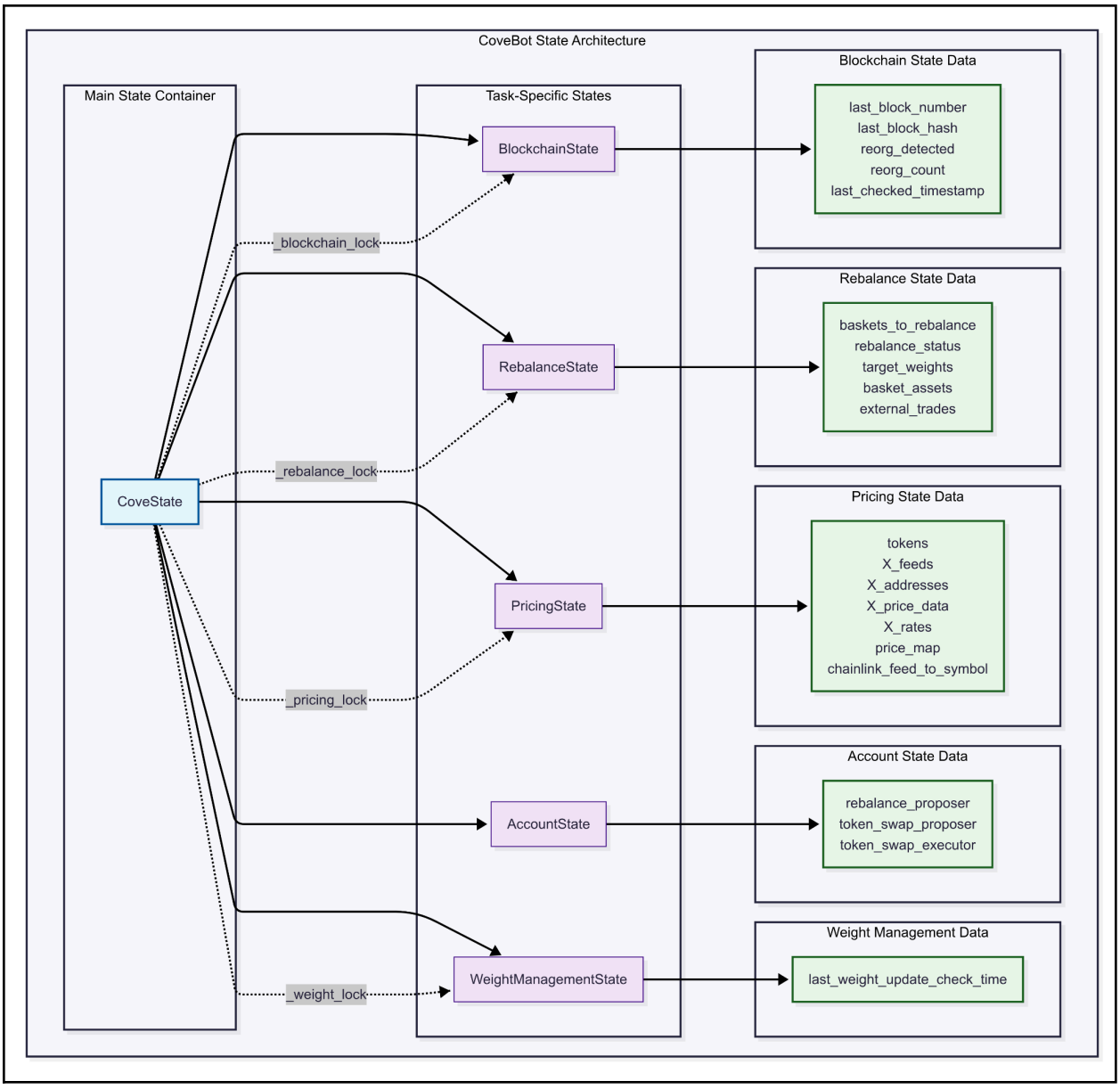


Figure 1: Architecture proposal

This will allow to

- **Task isolation**: the split of the scope clarifies the use of variables
- **Explicit variables**: to replace the usage of `SimpleNamespace`
- **Thread safety**: through thread locks
- **Simple API usage:** through the context manager (allowing to use the variable's lock through `with`)
- **Reorg detection**: through a saved blockchain state

State separation

Instead of having one `CoveState` or one `SimpleNamespace`, split the state into multiple classes. For example:

- **RebalanceState**: contains all the rebalance related variables (`baskets_to_rebalance`, `rebalance_status`, ...)
- **PricingState**: contains all the pricing related variables (feeds, price data etc)

Thread lock

Every class can be protected through a lock and a context manager, such that:

```
class CoveState:
   def __init__(self):
       # Individual locks for each state type
       self._rebalance_lock = threading.Lock()
       # ...

       # State instances
       self._rebalance = RebalanceState()
       # ...

   # Single context manager for each state type
   @contextmanager
   def rebalance(self):
       with self._rebalance_lock:
           yield self._rebalance
```

Figure 2: CoveState with lock per sub-class

This will allow to manipulate the objects through `with:`

```
def some_operation(bot):
   with bot.state.cove.pricing() as pricing:
```

Figure 3: Example of usage

Making it explicit and easy to manipulate thread safe objects.

Reorg tracking

A separate class can be added to track the risk of reorg. The class can track

- Latest block number
- A dictionary of block number and block hash

A re-org detection mechanism can then be implemented, by checking:

- If a block.number older than the latest block number
- If a block.number hash does not match the hash already seen

Which can then trigger an alert to the Storm Labs's team

Appendix G: Watching assets

The following contains ideas of additional checks/monitoring that can be implemented to reduce the risks associated with the assets integrated in Cove:

- **Contract upgrade.** An upgrade might change the integration requirements, or might be an indication of an incident.
- **Large shift to the total supply**. A large shift might be an indication of an incident. The following strategy can be used:
    - Mint or Burn events, if they are available
    - Large Transfer event from or to the 0x0 address
    - Or regular check to totalSupply() if none of the above methods work
- **Large price shift.** To prevent rebalancing actions during high volatility period
- **Large price deviation between sources.** As it might indicate a black swan event within the market
- **Paused actions.** If an asset is paused, it might break its overall integration within Cove
- **Enabling non-standard assets features.** For example if an asset has an optional transfer fee that is activated, it might impact the rebalancing process.

In addition, the following can be considered when integrating new assets:

- Is there an audit of the contracts?
- Are the assets covered by a bug bounty program?
- Are the owners of the contracts easily identified and publicly known?